Self-Healing and Intelligent Root Cause Analysis System for Data Link Faults in Cloud-Native Environments

Yingqiang Yuan
Virscend University
Irvine, CA, USA
yuanyingqiang05@163.com

Abstract

To address the insufficient self-healing capabilities and difficult root cause analysis of data link faults in cloud-native environments, this paper proposes an intelligent operation and maintenance system for data links. Based on eBPF technology, this paper achieves zero-intrusion collection of network golden indicators and dynamic service call topology construction. An operation and maintenance knowledge graph is built and multi-source data is fused. An intelligent root cause analysis algorithm combining graph neural networks and causal analysis is employed, reducing fault location time to the second level. Finally, a reinforcement learning self-healing decision engine based on deep Q-networks is designed to dynamically select the optimal recovery action, driven by a Kubernetes declarative API. Experimental results show that, in a thousand-node cluster, the system achieves an average root cause analysis time of 15.5 seconds and a Precision@1 of 0.78; the self-healing success rate is improved by 31.9%–90.8%, verifying the significant effectiveness of the proposed method in improving the resilience and intelligent operation and maintenance level of cloud-native systems.

CCS Concepts

• **Software and its engineering** → Software notations and tools; Software maintenance tools.

Keywords

Cloud-Native, Data Link, Root Cause Localization, Deep Q-Network

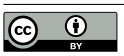
ACM Reference Format:
Yingqiang Yuan. 2026. Self-Healing and Intelligent Root Cause Analysis System for Data Link Faults in Cloud-Native Environments. In *2026 6th International Conference on Applied Mathematics, Modelling and Intelligent Computing (CAMMIC 2026), March 27–29, 2026, Shanghai, China.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3817124.3817267

1 Introduction

With the continuous evolution of cloud computing technology, cloud-native architecture, with Kubernetes at its core, has become the de facto standard for modern enterprise application deployment. While containerization, microservices, and dynamic orchestration provide systems with extreme elasticity and agility, they also bring unprecedented operational complexity. This paper proposes a fault self-healing and intelligent root cause localization system for data links in cloud-native environments. Its main contributions include three aspects: First, designing and implementing a zero-intrusion data acquisition mechanism based on eBPF, efficiently capturing key network metrics without modifying business code, and dynamically constructing service call topologies to lay a solid data foundation for upper-layer intelligent analysis. Second, proposing a hybrid root cause localization model integrating knowledge graphs, graph neural networks, and causal analysis, accurately pinpointing the source of faults through multi-source data association and causal inference, compressing the traditional troubleshooting process of several hours to seconds. Third, a self-healing decision engine based on deep reinforcement learning is introduced, modeling the fault recovery problem as a sequential decision problem. This enables the system to dynamically select the optimal combination of recovery strategies based on real-time status and reliably execute self-healing actions through Kubernetes native APIs.

2 Related Work

With the widespread application of cloud-native architecture, its dynamic, distributed characteristics and complexity pose serious challenges to the observability, elasticity and intelligent operation and maintenance capabilities of the system. Yang and Duan [1] designed a Kubernetes-based testing framework and realized multi-source data collection through service detection. Ren et al. [2] analyzed the key points of intelligent operation and maintenance architecture of cloud-native technology. Fu et al. [3] analyzed the correlation between three types of faults and multi-source data and constructed a dynamic multi-layer microservice architecture. Al-Said Ahmad et al. [4] proposed a fault detection method based on cloud-native architecture. Brahmandam and Chandramohan [5] discussed how to integrate artificial intelligence and AIOps frameworks into the cloud-native environment. Tadi [6] designed an API with autonomous fault recovery capability for event-driven microservice architecture. Beereddy [7] proposed that applying cloud-native fault tolerance technology in financial and billing platforms can significantly improve the system's elasticity to various faults without sacrificing data consistency and accuracy. Kumar [8] constructed an elastic engineering model suitable for cloud-native software systems. Oyeniran et al. [9] summarized cloud-native technologies for improving software scalability and resilience. Gangina [10] explained the principle of applying chaos engineering to distributed cloud-native applications to enhance their resilience. Existing work mainly focuses on fault management of application layer services or microservice architectures, while paying less attention to fault self-healing and intelligent root cause localization of data links that carry business flows. This paper aims to build a real-time fault detection, adaptive self-healing and accurate root



This work is licensed under a Creative Commons Attribution 4.0 International License.
CAMMIC 2026, Shanghai, China
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2329-2/2026/03
https://doi.org/10.1145/3817124.3817267

cause localization mechanism for data links to improve the overall resilience and observability of cloud-native systems.

3 Methods

3.1 *Data Acquisition and Link Topology Construction Based on eBPF*

Data collection is achieved by deploying a DaemonSet-style agent on each Kubernetes node. This agent loads and executes an eBPF program for non-intrusive data collection. The eBPF program intercepts all network packets entering and leaving the container by attaching TCP connection events and socket read/write entry points to the kernel via kprobes. The raw events collected are pre-aggregated by a percpu array in kernel space and then periodically pushed to user space via BPF mappings (default interval is 10 seconds).The pre-aggregation operation involves calculating the cumulative length of the data packets, S_{total}:

$$S_{total} = \sum_{i=1}^{n} \text{len}(pkt_i) \tag{1}$$

$\text{len}(pkt_i)$ is the byte length of the i-th data packet. The user-space processor associates the data with a four-tuple (source IP, source port, destination IP, destination port) and the Pod Label and Service information issued by the Kubernetes API Server, mapping the network five-tuple to service call relationships.

Based on this, the system automatically refreshes the global service call topology graph every 30 seconds. Nodes in the graph represent Services, edges represent call relationships, and four key metrics are appended to each edge: request rate (req / sec), latency distribution (P50, P90, P99), error rate (5xx/all requests), and saturation (TCP retransmission rate and connection queue backpressure event count). This mechanism does not rely on business code instrumentation, is uniformly effective for multi-language applications [11-12].

This observability assessment is based on the following core assumptions:

Plaintext Traffic Assumption: The current implementation primarily targets unencrypted traffic. For TLS/SSL encrypted communication (HTTPS), the eBPF program can only see the encrypted packets and cannot parse the application layer protocol headers, thus failing to extract information such as service names and status codes.

Protocol Standardization Assumption: The protocol parser is implemented based on standard protocol specifications (RFCs). For highly customized or non-standard traffic using proprietary protocols, parsing may fail, resulting in the inability to correctly identify and label those call relationships.

Sidecar Transparency Assumption: In a Sidecar deployment mode using Service Mesh such as Istio, business traffic is transparently redirected to the Sidecar proxy (Envoy). The eBPF program in this system is mounted in the host kernel mode, enabling complete observation of all data packets from the business container to the Sidecar, and from the Sidecar to downstream services. Since communication occurs within the same node, potentially using Unix Domain Sockets or loopback interfaces, this increases the complexity of traffic association.

3.2 *Intelligent Root Cause Localization Based on Knowledge Graph*

Intelligent localization first constructs an operation and maintenance knowledge graph, and integrates multimodal data such as dynamic topology, Prometheus metrics, Kubernetes Events and container logs output by the collection layer into a unified graph structure. The entity types include six categories: microservices, containers, Pods, nodes, middleware instances and APIs, and the relationship types include eight types such as “deployed in”, “call”, “connection” and “containment”. For unstructured logs, the BERT-BiLSTM-CRF model is used to extract entities and associate variables such as exception stacks in the log template with entities. This graph not only stores the system’s static configuration information, but more importantly, it integrates real-time dynamic indicators and events, providing rich context for subsequent analysis.

When a fault occurs, the exception subgraph within the fault time window is extracted in real time and input into the graph neural network model [13]. The model outputs the probability embeddings of each entity as the root cause of the failure. The aggregation process can be formalized as the calculation of the mean of the neighborhood features:

$$h_v^{(k)} = \sigma\left(\text{Wmean}\left(\left\{h_v^{(k1)}\right\}\left\{h_u^{(k1)}, \forall u \in N(v)\right\}\right)\right) \tag{2}$$

$h_v^{(k)}$ is the feature vector of node v at the k-th layer, N(v) is the set of neighboring nodes of node v, W is the trainable weight matrix, and σ is the activation function. The model outputs the probability embeddings of each entity as the root cause of the failure, and the recall metric is set to recall the top 20 candidate entities [14-15]. The goal of this stage is not precise localization, but to leverage the powerful structural information perception capabilities of GNNs to quickly filter out a small set of candidates that are most likely the root cause from thousands of entities, thereby reducing the burden on subsequent causal analysis, which has a higher computational complexity.

Subsequently, the system constructs a causal graph: Granger causality tests are performed on time-series indicators using the PC algorithm to filter out causal pairs and generate directional causal edges. The Granger causality test is based on the comparison of vector autoregressive models:

$$F = \frac{(RSS_r - RSS_{ur})/p}{RSS_{ur}/(T - 2p - 1)} \tag{3}$$

F is the test statistic, RSS_r is the sum of squared residuals of the constrained model (excluding lagged independent variables), RSS_{ur} is the sum of squared residuals of the unconstrained model (including lagged independent variables), p is the lag order, and T is the sample size. The causal graph constructed in this way is no longer a simple copy of the service call relationship, but reflects the inherent, temporally sequential driving relationship between the indicators of each entity when a failure occurs.

The candidate set output by the GNN is used as nodes in the causal graph, and the PageRank algorithm is used for iterative sorting. The damping coefficient is set to 0.85, and the maximum number of iterations is 50, until convergence. The PageRank value

of each node is updated as follows:

$$PR(v) = \frac{1-d}{N} + d \sum_{u \in M(v)} \frac{PR(u)}{L(u)} \quad (4)$$

PR(v) is the PageRank score of node v, d is the damping coefficient (set to 0.85), N is the total number of nodes in the graph, M(v) is the set of nodes pointing to node v, and L(u) is the number of outgoing links to node u. The algorithm outputs the final probability score of each candidate root cause. Entities with a score higher than 0.65 are judged as primary root causes.

3.3 *Self-Healing Decision and Execution Based on Reinforcement Learning*

The self-healing decision engine encodes the fault location results and the current cluster state into a reinforcement learning state space. The state vector is composed of three parts: one-hot encoding of fault types; root cause entity attributes such as current CPU utilization, request QPS, and error rate; and the cluster's global water level (total remaining CPU/memory, number of pending Pods). The action space is defined as pre-defined recovery actions in the policy pool, including four main categories of atomic actions: elastic scaling (increasing or decreasing the number of HPA replicas, adjusting resource limits), service degradation (circuit breaker switches, traffic coloring degradation), traffic governance (modifying Istio DestinationRule weights for traffic switching), and infrastructure self-healing (rebuilding Pods, restarting Sidecars). The reinforcement learning algorithm uses a deep Q-network (DQN). The Q-network structure is a three-layer fully connected network (input layer 256, hidden layer 128, output layer 15). The learning rate is set to 0.001, the experience replay pool capacity is 10000, and the exploration rate ϵ starts from 1.0 and decays to 0.01 with the number of training steps. The reward function R is designed as a composite form: if the fault index recovers to below the threshold within a preset time window (2 minutes) after the self-healing action is executed, the reward value is set to +1; if it recovers partially, a reward between 0 and 1 is given according to the recovery ratio. If the fault worsens or remains unchanged, the reward is -1; if the action involves destructive operations, a penalty term of -2 is introduced to encourage conservative strategies.

4 Results and Discussion

4.1 *Experimental Environment and Dataset*

To ensure the completeness and reproducibility of the dataset, for six types of network faults (ARP cache pollution, VLAN configuration errors, MTU configuration errors, VXLAN tunnel encapsulation errors, CNI network plugin failures, and MAC address conflicts), each fault was repeatedly injected 200 times at random intervals. After each injection, metrics were continuously monitored until recovery or a 10-minute timeout occurred, resulting in 2000 valid fault samples. Each sample includes the key metrics collected by eBPF within the fault time window, Kubernetes Events, Prometheus time-series data, and corresponding root cause labels (confirmed jointly by manual annotation and chaos experiment records). The samples were divided into training (70%), validation (15%), and test (15%) sets in chronological order to ensure that the model was evaluated on unseen data.

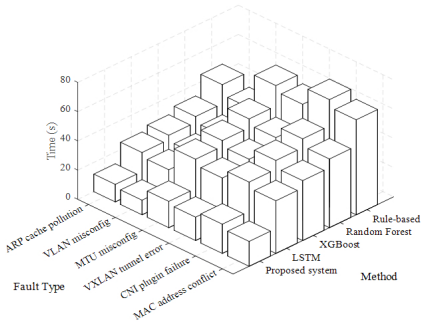


Figure 1: Comparison of MTTR for different methods under various fault types (unit: seconds)

4.2 *Root Cause Localization Performance Comparison*

Four mainstream baseline methods were selected: rule-based alarm compression, Random Forest, XGBoost and the LSTM-based time series anomaly detection. All methods were trained and tested on the same data set of fault injection. The fault types cover six typical faults in cloud-native environments: ARP cache pollution, VLAN configuration errors, MTU configuration errors, VXLAN tunnel encapsulation errors, CNI network plug-in failures, and MAC address conflicts. The experiment was repeated 5 times and the average was taken to ensure robustness of the results.

4.2.1 Fault Localization Time (MTTR) Comparison. Figures 1 shows reflect averaged localization for different methods for various fault types. The data indicates that we get the MTTR of our system much less than the baseline methods for all the types of faults.

The proposed method achieves an average root cause localization time of 15.5 seconds across all fault types, a 72.5% reduction compared to rule-based methods (56.3 seconds) and a 52.3% reduction compared to the best-performing baseline method, LSTM (32.5 seconds). In the MTU configuration error scenario, the proposed method completes root cause localization in just 18 seconds, while LSTM requires 38 seconds and rule-based methods take as long as 62 seconds.

4.2.2 Comparison of Precision and Recall. The accuracy of root cause localization is measured by Precision@k (k=1,3,5) and overall recall (the proportion of the root cause appearing in the Top-20 candidates). Figure 2 shows the metrics for each method.

The proposed system achieves a Precision@1 of 0.78, a 41.8% improvement over LSTM (0.55) and a 143.75% improvement over the rule-based method (0.32). As the candidate set expands, Precision@5 reaches 0.96, meaning that the root cause has a 96% probability of appearing in the top 5 candidate entities output by the system. In terms of recall, the proposed system significantly surpasses LSTM's 0.80 and the rule-based method's 0.58 with a recall of 0.94, validating that the causal reasoning capability of knowledge graphs combined with GNNs can effectively capture the full propagation of complex faults.

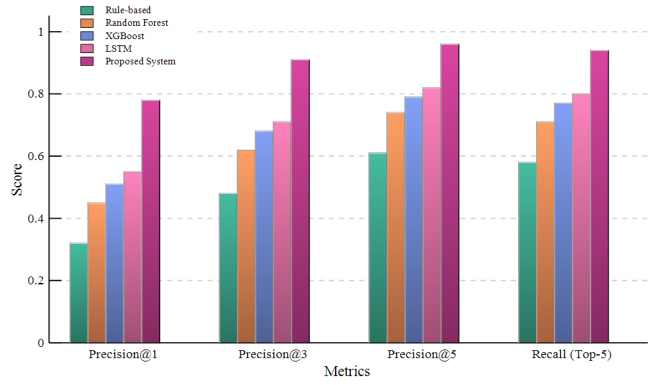


Figure 2: Comparison of Precision and Recall of Different Methods

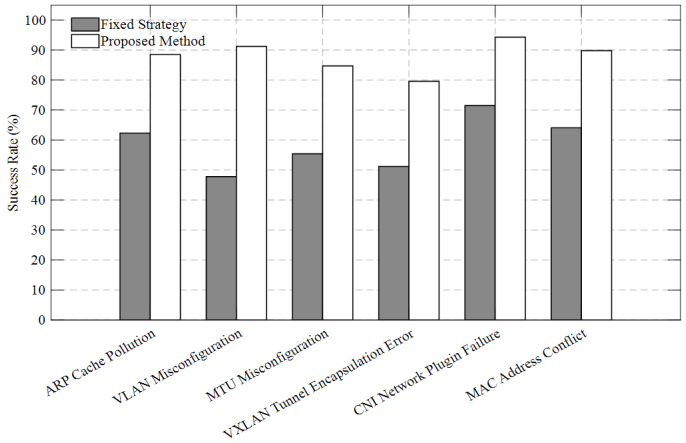


Figure 3: Comparison of self-healing success rates under different fault types

4.3 *Comparison of Self-Healing Success Rate*

For six typical fault types, each type was repeatedly injected 100 times, and the self-healing success rate (business metrics recovering to within the threshold within 5 minutes) was recorded. The fixed strategy script is based on threshold rules: when CPU > 80%, replicas are expanded; when the error rate > 5%, circuit breaker degradation is activated. Figure 3 summarizes the comparison results of the two methods:

The RL engine demonstrated significantly higher self-healing success rates across all fault types compared to fixed strategies, particularly achieving a 90.8% success rate in VLAN configuration errors. This is attributed to the RL engine's ability to dynamically select the optimal action combination based on real-time status, while fixed strategies rely solely on a single threshold trigger, making them prone to local optima. In CNI network plugin failures, the RL engine analyzed CNI plugin logs and node network status to execute combined route optimization and connection reconstruction actions, achieving a 94.3% success rate. The success rate improvement fluctuated between 31.9% and 90.8%, reflecting the RL engine's adaptability to complex fault modes.

4.4 *System Overhead and Scalability*

On a single node, the wrk tool is used to send incremental HTTP requests to the target service, increasing the rate from 1000 to 5000 req/s, with each rate running for 10 minutes. The CPU and memory usage of the container hosting the Agent were sampled every minute using the Prometheus Node Exporter, and the P99 latency for GNN root cause localization during fault injection was recorded (triggered 100 times in total). Table 1 summarizes the key test results:

Memory usage increased steadily from 128 MB to 215 MB, primarily due to the expansion of the kernel percpu array and BPF maps. GNN inference latency increased with the request rate, from 23 ms to 44 ms, validating the model's inference efficiency.

Six clusters of different sizes (1000, 1200, 1400, 1600, 1800, and 2000 nodes respectively) were built, with a base cluster of 1000

Table 1: eBPF Data Acquisition Module and GNN Inference Overhead

QPS (req/s)	eBPF CPU usage (%)	eBPF memory usage (MB)	GNN inference latency (ms)
1000	2.3	128	23
2000	3.8	156	28
3000	4.9	172	31
4000	6.7	191	37
5000	8.2	215	44

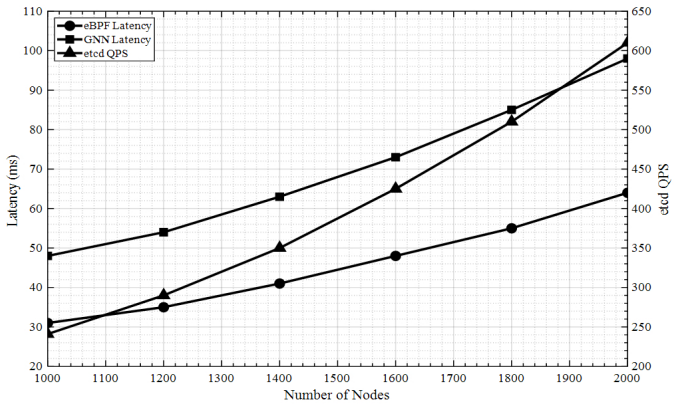


Figure 4: Performance Changes with Cluster Size Expansion

nodes and gradually increased to 2000 worker nodes. Each cluster deployed the same online marketplace microservice application with a fixed ingress traffic of 2000 req/s. The eBPF acquisition latency is defined as the time difference between packet capture from the kernel and writing to the metrics store. GNN inference latency is recorded by the root cause localization module, and etcd QPS increment is derived by comparing the etcd query rate before and after system operation. Figure 4 shows the trends of these metrics with the number of nodes:

The eBPF acquisition latency increased from 31 ms with 1000 nodes to 64 ms with 2000 nodes, an increase of approximately 106.5%, mainly due to data processing queue backlog caused by the increased number of kernel events. GNN inference latency increased from 48 ms to 98 ms, reflecting the computational pressure brought by the increased graph size. Although all metrics increased at 2000 nodes, acceptable performance was maintained, indicating that the proposed method has good horizontal scalability.

5 Conclusion

This paper achieves low-overhead network golden index acquisition through eBPF technology, compresses fault location time to the second level based on a fusion model of knowledge graph and graph neural network, and utilizes deep reinforcement learning to drive self-healing decisions, improving the recovery success rate in complex fault scenarios. However, this study still has certain limitations: the offline training of the reinforcement learning model relies on historical fault data, and its generalization ability to unseen fault patterns needs improvement; causal analysis suffers from computational latency in highly dynamic microservice graphs. Future work can explore few-shot fault diagnosis based on meta-learning, introduce online learning mechanisms to achieve continuous policy evolution, and further optimize real-time inference performance on large-scale graphs.

References

[1] Yang Yue, Duan Panpan. Research on testing technology of cloud-native IT system based on hybrid diagnostic model [J]. Software, 2025, 46(11): 131-133.
[2] Ren Gaoqiang, Cui Yunlong, Gao Ping, Lü Shulin, Zhao Junsheng. Intelligent operation and maintenance architecture and key technologies for cloud-native [J]. China New Communications, 2021, 23(12): 46-47.
[3] Fu Nan, Cheng Guang, Dai Guangye, Teng Yue. Fault diagnosis method for microservice system for cloud-native network [J]. Science in China: Information Science, 2025, 55(8): 1888-1905.
[4] Al-Said Ahmad A, Al-Qora'n L F, Zayed A. Exploring the impact of chaos engineering with various user loads on cloud native applications: an exploratory empirical study[J]. Computing, 2024, 106(7): 2389-2425.
[5] Brahmandam B A, Chandramohan S. Using artificial intelligence and AIOps, automated fault prediction and prevention in Cloud Native settings[J]. International Journal of Computer Techniques, 2024, 11(6): 1-7.
[6] Tadi S. Architecting resilient cloud-native apis: Autonomous fault recovery in event-driven microservices ecosystems[J]. Journal of Scientific and Engineering Research, 2022, 9(3): 293-305.
[7] Beereddy K R. Cloud-Native Resilience Engineering: Fault Tolerance in Financial and Billing Platforms[J]. Journal Of Engineering And Computer Sciences, 2025, 4(8): 431-436.
[8] Kumar M. Resilience Engineering Models for Cloud-Native Software Systems[J]. International Journal of Science, Research and Technology, 2024, 7(4): 12424-12433.
[9] Oyeniran O C, Modupe O T, Otitoola A A, *et al.* A comprehensive review of leveraging cloud-native technologies for scalability and resilience in software development[J]. International Journal of Science and Research Archive, 2024, 11(2): 330-337.

[10] Gangina P. Resilience engineering principles for distributed cloud-native applications under chaos[J]. International Journal of Computer Technology and Electronics Communication, 2022, 5(5): 5760-5770.
[11] Putrevu A. Reinforcement Learning-Driven Fault Recovery in Cloud-Native Data Integration Architectures[J]. Journal of Computer Science and Technology Studies, 2025, 7(9): 508-515.
[12] Żurkowski B, Zieliński K. Root cause analysis for cloud-native applications[J]. IEEE Transactions on Cloud Computing, 2024, 12(1): 232-250.
[13] Purbaratri W, Mujito M, Al Din S J. A Comparative Study of Software Testing Techniques and Quality Metrics for Predicting Failure Rates in Scalable Cloud Native Software Systems[J]. Software Engineering in Computing Systems, 2026, 1(1): 41-51.
[14] Deng S, Zhao H, Huang B, *et al.* Cloud-native computing: A survey from the perspective of services[J]. Proceedings of the IEEE, 2024, 112(1): 12-46.
[15] Agrawal P. Microservices architecture: A modern approach to cloud-native development[J]. Journal of Computer Science and Technology Studies, 2025, 7(6): 709-716.